

# pse-edge-mcp — Developer Walkthrough

**Audience:** a developer or architect who has never seen this codebase and needs to understand it, debug it, or extend it.

**Companion documents.** This file explains *how the system works*. Two others explain *why it is the way it is*, and you should read them before changing anything structural:

| Document          | Contents                                                                                |
|-------------------|-----------------------------------------------------------------------------------------|
| docs/plan.md      | Every design decision: scope, caching policy, auth design, phases, risks                |
| docs/endpoints.md | The verified PSE Edge endpoint map — request dialects, parameter names, response shapes |
| docs/deploy.md    | Running it in production                                                                |
| CLAUDE.md         | The short form of the invariants, kept next to the code                                 |

Version described: **0.8.1**. 34 modules, ~7,300 lines of source, ~5,250 lines of tests.

## 1. What this is, in one paragraph

An **MCP server** (Model Context Protocol — a standard way to expose tools to an LLM client) that serves **Philippine Stock Exchange** data taken from the PSE Edge disclosure portal. It exposes 12 read-only tools: quotes, price history, disclosures, financial reports, dividends, and index/market data.

It is **unofficial**. PSE Edge publishes no API, so this server speaks to the same internal endpoints the portal's own web pages call.

### The one decision that explains the whole design

PSE Edge is a public utility with no rate-limit contract and no commercial relationship with this project. Hammering it during trading hours would be antisocial and would get the project blocked. So:

**This is an end-of-day server. It makes zero upstream requests while the market is open.**

Almost every structural choice follows from that sentence — the cache is a *freeze* rather than a TTL, every response carries freshness metadata, there is an opportunistic archive, and there is a politeness throttle independent of user quotas. If a change you are contemplating would increase load on PSE Edge, it is probably the wrong change.

## 2. Running it

Three modes, same tool implementations underneath.

| MODE                 | COMMAND                                                           | AUTH   | STORAGE                  |
|----------------------|-------------------------------------------------------------------|--------|--------------------------|
| studio<br>(local)    | uvx pse-edge-mcp<br>(Claude Desktop / Claude Code default)        | never  | in-memory                |
| HTTP<br>(dev)        | pse-edge-mcp --http --port 8000                                   | opt-in | in-memory or<br>Postgres |
| HTTP<br>(production) | docker compose -f compose.nas.yaml up -d<br>+ compose.tunnel.yaml | on     | Postgres                 |

**studio never authenticates.** It runs on the user's own machine as a subprocess of their client; there is no network boundary to guard and no second party to authenticate. Adding auth there would be pure friction.

Local development:

```
uv sync --all-extras      # Python 3.14, uv for everything
uv run pytest              # 271 tests, no network access
uv run ruff check .        # line length 100
uv run mypy src            # strict
```

### 3. Request lifecycle

This is the path every tool call takes. Read it once and most of the codebase becomes predictable.

MCP client (Claude Desktop, claude.ai connector, your own agent)

POST /mcp {"jsonrpc":"2.0","method":"tools/call", ...}

|                |                                                                                                                                  |
|----------------|----------------------------------------------------------------------------------------------------------------------------------|
| HealthApp      | /health, /health/ready – answered here, never authenticated, never touch the DB                                                  |
| AuthApp        | /oauth/*, /signup, /login, /account, /privacy, / – reachable WITHOUT a token, because you cannot present one before you have one |
| AuthMiddleware | bearer token → user; quota check; 401 + WWW-Authenticate, or 429                                                                 |
| MCP app (SDK)  | JSON-RPC framing, tool dispatch, DNS-rebinding Host check                                                                        |

server.py validate args → call repository → shape reply  
(no domain logic here – see §5)

repositories.py build cache key → FreezeService.get(...) → parse → model

service.py  
FreezeService

|                                                                         |
|-------------------------------------------------------------------------|
| THE FREEZE DECISION – see §4<br>cache hit? market open? fetch or refuse |
|-------------------------------------------------------------------------|

cache hit

Storage  
(memory | Postgres)

cache miss + closed

client.py —HTTP→ edge.pse.com.ph  
parsers.py ← HTML / JSON

archive.py (opportunistic, best-effort)

{"data": ..., "meta": {as\_of, valid\_until, from\_cache, stale, data\_policy}}

Two things worth noticing:

- **The composition is built exactly once**, in `asgi.py`. The CLI (`__main__.py`) and production both call `create_app()`, so they cannot drift apart.
- **/mcp is the only authenticated path**. Everything the browser needs during signup is deliberately in front of the auth gate.

---

## 4. The freeze policy — the core behaviour

This is the part to understand before anything else, because it is what makes the server *correct* rather than merely functional.

`FreezeService.get()` in `service.py` implements one decision table:

|                          | MARKET CLOSED                   | MARKET OPEN                                                                          |
|--------------------------|---------------------------------|--------------------------------------------------------------------------------------|
| cache fresh              | serve from cache                | serve from cache                                                                     |
| cache stale<br>(expired) | FETCH upstream, store,<br>serve | serve the stale value,<br>flagged <code>meta.stale = true</code><br>← never fetches  |
| cache empty              | FETCH upstream, store,<br>serve | raise <code>MARKET_OPEN_NO_CACHE</code><br>with a <code>retry_after</code> timestamp |

Market hours are **09:30–15:00 Asia/Manila on trading days**; weekends and the `PSE_HOLIDAYS` table in `market_calendar.py` are non-trading.

### Consequences a newcomer will trip over

- **`MARKET_OPEN_NO_CACHE` is not a bug.** It is the policy working. Agents must handle it; a `retry_after` timestamp is included so they can schedule rather than poll.
- **`meta.stale = true` means the market is open** and you are seeing the last close. It does not mean the data is wrong.
- **Concurrent cache misses collapse into one upstream request** (`SingleFlight` in `ratelimit.py`). Ten simultaneous first-time requests for the same symbol produce one HTTP call, not ten.
- **`immutable=True`** (passed by the disclosure-detail path) marks objects that never change upstream — a disclosure identified by `edge_no`. Those are cached forever with `valid_until: null` and `data_policy: "immutable"`. The open-market freeze still governs their *first* fetch: protecting PSE Edge outranks serving a cache miss promptly.

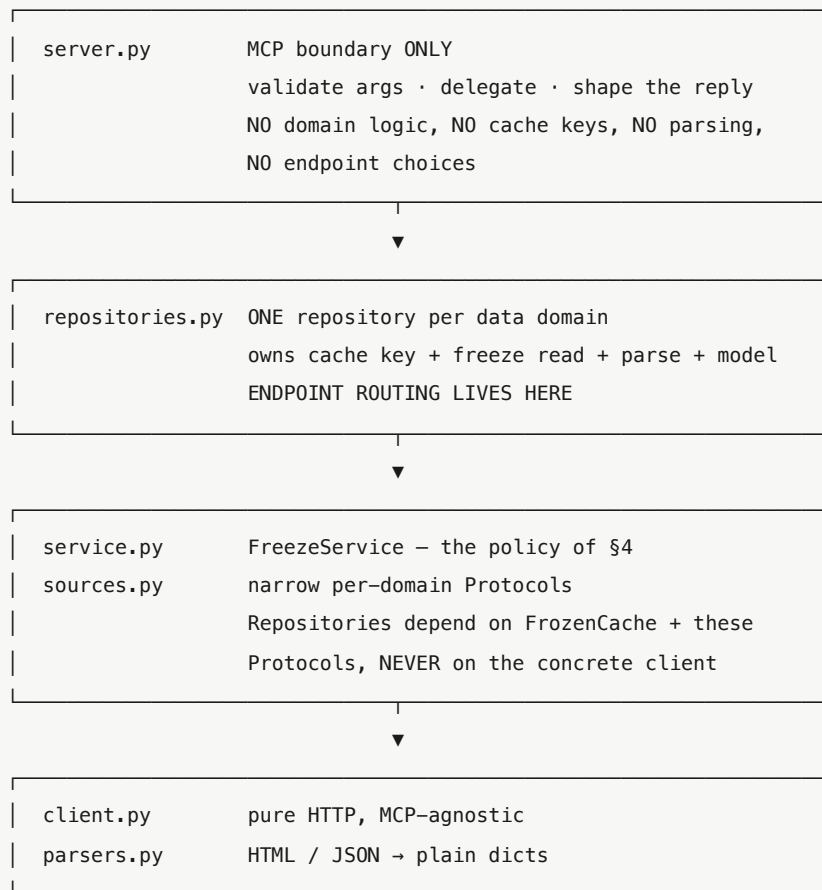
### The politeness throttle is separate from quotas

`throttle_rate_per_sec` (default 1.0/s, burst 2) limits **outbound** requests to PSE Edge and exists to protect *them*. Per-user quotas (60/min, 2000/day) limit **inbound** requests and exist to protect *you*. Do not conflate them; raising one does not justify raising the other.

---

## 5. Architecture and layering

The layering is enforced by convention and by tests. Violating it is the most likely way a well-meaning change gets rejected in review.



Supporting modules: `validation.py` (argument validators), `models.py` (Pydantic), `errors.py`, `cache.py` (Storage protocol), `archive.py`, `ratelimit.py`, `market_calendar.py`, `config.py`, `memo.py`.

### Why repositories depend on Protocols

`sources.py` defines five narrow interfaces — `CompanySource`, `QuoteSource`, `DisclosureSource`, `CompanyInfoSource`, `MarketSource`. `PseEdgeClient` happens to satisfy all of them, but repositories only ever see the narrow one they need. The payoff is in `tests/test_repositories.py`: a repository can be tested with a five-line fake and **no HTTP mocking at all**.

## Module map

| Module                       | Lines | Responsibility                                                                                |
|------------------------------|-------|-----------------------------------------------------------------------------------------------|
| <code>auth_app.py</code>     | 756   | Browser- and client-facing routes: OAuth, signup, login, account, privacy                     |
| <code>parsers.py</code>      | 752   | HTML/JSON → dicts. The most PSE-specific code in the repo                                     |
| <code>oauth.py</code>        | 721   | OAuth 2.1 server: DCR, PKCE, code exchange, refresh rotation, <code>client_credentials</code> |
| <code>repositories.py</code> | 526   | The domain layer — five repositories                                                          |
| <code>admin.py</code>        | 431   | <code>pse-edge-admin</code> CLI, including machine-client provisioning                        |
| <code>passkeys.py</code>     | 424   | WebAuthn ceremonies and browser sessions                                                      |
| <code>server.py</code>       | 413   | The 12 tool definitions                                                                       |
| <code>models.py</code>       | 332   | 24 Pydantic models                                                                            |
| <code>client.py</code>       | 268   | HTTP to PSE Edge; both request dialects                                                       |
| <code>db.py</code>           | 255   | SQLAlchemy tables, engine, schema check                                                       |
| <code>asgi.py</code>         | 240   | The single composition point, and the DNS-rebinding allowlist                                 |
| <code>auth.py</code>         | 190   | <code>TokenService</code> , <code>QuotaTracker</code> . Must stay SQLAlchemy-free             |

## 6. The tool surface

All 12 tools are read-only and return the same envelope (§7).

| Tool                                    | Arguments                                                                                                                                               | Repository                                              |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <code>search_companies</code>           | <code>query</code>                                                                                                                                      | <code>CompanyRepository.search</code>                   |
| <code>validate_symbol</code>            | <code>symbol</code>                                                                                                                                     | <code>CompanyRepository.try_resolve</code>              |
| <code>get_stock_quote</code>            | <code>symbol</code>                                                                                                                                     | <code>QuoteRepository.quote</code>                      |
| <code>get_price_history</code>          | <code>symbol</code> , <code>start_date?</code> ,<br><code>end_date?</code>                                                                              | <code>QuoteRepository.history</code>                    |
| <code>search_disclosures</code>         | <code>symbol?</code> , <code>start_date?</code> ,<br><code>end_date?</code> , <code>template?</code> ,<br><code>page</code>                             | <code>DisclosureRepository.search</code>                |
| <code>search_disclosure_fulltext</code> | <code>keyword</code> , <code>start_date?</code> ,<br><code>end_date?</code> , <code>symbol?</code> ,<br><code>subject_title?</code> , <code>page</code> | <code>DisclosureRepository.fulltext</code>              |
| <code>get_disclosure</code>             | <code>edge_no</code>                                                                                                                                    | <code>DisclosureRepository.detail</code>                |
| <code>get_company_profile</code>        | <code>symbol</code>                                                                                                                                     | <code>CompanyInfoRepository.profile</code>              |
| <code>get_financial_highlights</code>   | <code>symbol</code>                                                                                                                                     | <code>CompanyInfoRepository.financials</code>           |
| <code>get_dividends_and_rights</code>   | <code>symbol</code>                                                                                                                                     | <code>CompanyInfoRepository.dividends_and_rights</code> |
| <code>get_indices</code>                | —                                                                                                                                                       | <code>MarketRepository.indices</code>                   |
| <code>get_market_summary</code>         | —                                                                                                                                                       | <code>MarketRepository.summary</code>                   |

Dates are YYYY-MM-DD on the tool surface; the client converts to PSE Edge's MM-dd-yyyy wire format.

`search_disclosures` routes to two different upstream endpoints depending on whether `symbol` is supplied — market-wide with a date range, or one company's full history. That routing decision lives in the repository, which is exactly where the layering says it belongs.

`search_disclosure_fulltext` is deliberately a separate tool, not a better search. PSE Edge's keyword index only covers roughly 2023–2025, so it is not a superset of `search_disclosures`. The tool reports this coverage limit in its own results.

## 7. The data contract

### Every tool returns this envelope

```
{
  "data": { "...": "the payload" },
  "meta": {
    "as_of":      "2026-07-31T15:00:00+08:00",
    "valid_until": "2026-08-01T15:00:00+08:00",
    "from_cache": true,
    "stale":      false,
    "data_policy": "EOD-frozen"
  }
}
```

`meta` is not decoration. It is the mechanism by which the server is honest about a freeze policy that would otherwise look like stale data. A client that ignores `meta` will eventually mislead someone about when a price was true.

- `data_policy: "immutable"` with `valid_until: null` marks objects that never change upstream.
- Clients should cache against `valid_until`.** Re-querying before it passes returns a byte-identical answer and only burns quota.

### Error codes

Errors are returned as data — `{"error": "CODE", "message": ..., ...}` — not as exceptions, so an LLM can react to them rather than seeing a traceback.

| Code                 | Meaning                               | Caller should                        |
|----------------------|---------------------------------------|--------------------------------------|
| MARKET_OPEN_NO_CACHE | Nothing cached and the market is open | Retry after <code>retry_after</code> |
| SYMBOL_NOT_FOUND     | No such ticker                        | Try <code>search_companies</code>    |
| INVALID_ARGUMENT     | Failed validation                     | Fix the arguments                    |
| ENDPOINT_CHANGED     | PSE Edge's response shape changed     | <b>Alert a human — see §11</b>       |
| EDGE_UNAVAILABLE     | Upstream unreachable or erroring      | Retry later                          |
| INTERNAL_ERROR       | Anything else                         | File a bug                           |

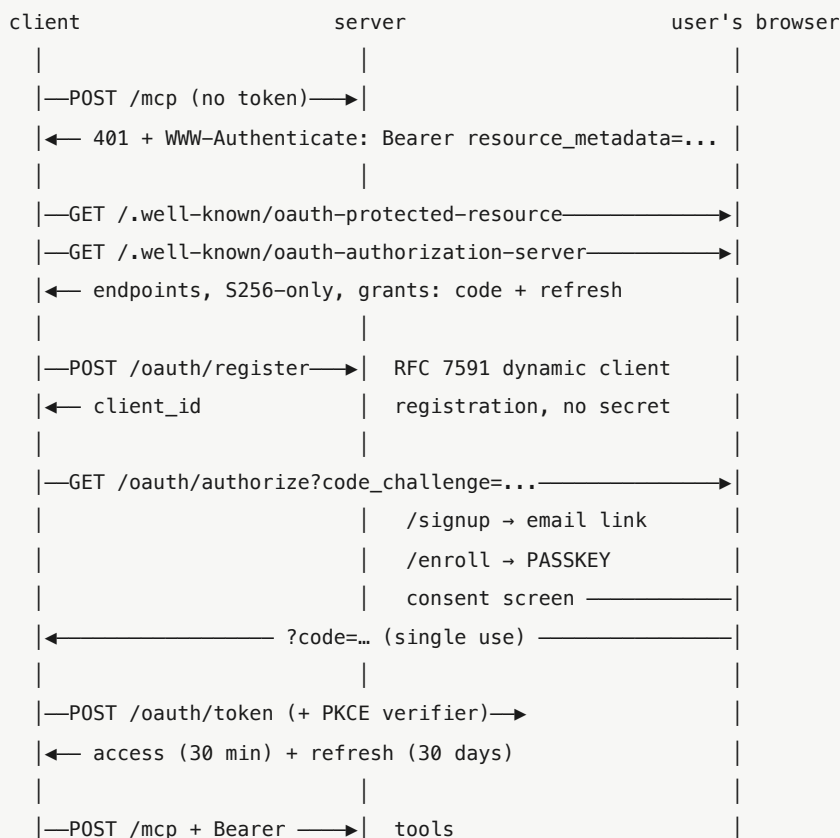
The mapping happens once, in `server.reply()`. Do **not** add per-tool `try/except`.

`reply()` is a helper that takes a callable, and deliberately not a decorator. The MCP SDK builds each tool's output schema from its return annotation, and a `functools.wraps` wrapper makes `inspect.signature` follow `__wrapped__` to the *inner* annotation, breaking schema generation. Tools must stay annotated `-> dict[str, Any]`. `tests/test_server_disclosures.py::test_tool_surface_is_stable` guards this.

## 8. Authentication

Auth is **opt-in** (`PSE_AUTH_REQUIRED=1`), which requires `DATABASE_URL`). `stdio` never authenticates.

### The self-service flow



Exactly two steps involve a human: the passkey enrollment and the consent click. No passwords exist anywhere in the system.

### Rules that must not regress

- Unknown client or unregistered redirect URI is **fatal and never redirects** — otherwise the server is an open redirector.
- Redirect URIs match **exactly**. No prefix matching.
- PKCE is mandatory, S256 only.**
- Authorization codes are single-use via one atomic `UPDATE ... WHERE consumed_at IS NULL RETURNING`.
- Refresh tokens rotate; **replaying a rotated one revokes the entire family** (RFC 9700 §4.14). A stolen refresh token gets one use before the theft is detected.
- Only `kind='access'` authenticates as a bearer.
- Tokens are opaque, `pse_`-prefixed, stored **only as SHA-256**.



- The validation-cache TTL (default 60 s) is **the revocation-latency budget and nothing else**. Refusals are never cached.
- `auth.py` must stay **SQLAlchemy-free** so the lean install path works; Postgres code lives in `auth_store.py` and imports lazily.

## Headless and agentic access — `client_credentials`

A LangGraph app or the Anthropic Messages API MCP connector cannot open a browser, so it exchanges a client id and secret for a bearer token directly:

```

agent —POST /oauth/token—→ grant_type=client_credentials
    |
    | HTTP Basic, or client_id/client_secret in the body
    |
← access_token (1 h), token_type=Bearer, scope=mcp
    |
    | ↑ NO refresh token
    |
—POST /mcp + Bearer —→ tools

```

Provisioned out of band; this is deliberately not reachable over HTTP:

```

pse-edge-admin create-machine-client --name langgraph-app # secret shown ONCE
pse-edge-admin revoke-machine-client <client_id>

```

**The security property to understand before touching any of this:** `/oauth/register` is open to the internet, so the right to use `client_credentials` must not be derivable from anything a registrant supplies — not a `grant_types` array it declares, not a requested auth method, not the presence of a secret. It is gated on `oauth_clients.client_type == 'machine'`, a column **only the admin CLI writes**. A DCR client that registers, declares the grant and sends a secret still gets `unauthorized_client`. If that check ever becomes conditional on request data, anyone on the internet can mint tokens.

Two more decisions worth knowing before changing them:

- **A machine client is backed by a service user** (under the reserved, non-routable `machine.invalid` domain). That is what keeps the bearer path identical for both grants: `/mcp` validates by joining `auth_tokens` to `users`, so a token with no user behind it would need a special case in the middleware — and a second path through authentication is where a revocation check goes missing later. It also gives machine clients quotas, usage accounting and disablement for free.
- **No refresh token for this grant.** The client already holds a long-lived secret it can present again; a refresh token would be a second credential of equal power without the rotation benefit that justifies one.

`POST /oauth/token` is rate-limited (`FixedWindowLimiter` in `ratelimit.py`, 20/minute per `client_id` **and** per IP), because it is the one endpoint where a long-lived credential can be attacked online. Both keys are always counted, so tripping one cannot keep the other cold.

The older route still exists for clients that speak neither OAuth nor this grant, and is the only option on a plain-HTTP deployment since browsers restrict WebAuthn to secure contexts:

```

pse-edge-admin create-user agent@example.com
pse-edge-admin issue-token agent@example.com --note nightly-job # shown once

```

Either way, give each agent **its own client or user**: quotas are per user, so a runaway job throttles itself, and revocation is surgical.

## Privacy surface

`/account` shows a signed-in user everything held about them; `POST /account/delete` erases it immediately, in one transaction, with no approval step. Usage is aggregated **per user-hour, never per request** — minimal collection *and* no write on the hot path — and purged after 90 days.

`tests/test_privacy.py::test_erasure_leaves_nothing_behind` walks `metadata.tables`, so a new user-keyed table **fails that test** rather than silently retaining personal data. If you add a table with a user foreign key, that test is your reminder to wire it into erasure.

## 9. Storage and the archive

Postgres is **optional**. `DATABASE_URL` unset gives an in-memory cache and a `NullArchive` — the zero-config stdio path, which does not even need the `postgres` extra installed.

| <code>DATABASE_URL</code> unset | <code>DATABASE_URL</code> set                   |
|---------------------------------|-------------------------------------------------|
| <code>InMemoryStorage</code>    | <code>PostgresStorage</code> → shared cache, so |
| <code>NullArchive</code>        | <code>N</code> replicas still cause exactly one |
| (per-process, ephemeral)        | upstream fetch per boundary                     |
|                                 | <code>PostgresArchive</code> → EOD bars and     |
|                                 | disclosures accumulate over time                |

Tables: `cache_entries`, `eod_bars`, `disclosures`, `users`, `auth_tokens`, `webauthn_credentials`, `web_sessions`, `oauth_clients`, `oauth_flows`, `email_verifications`, `usage_events`.

Two columns carry more weight than their size suggests. `oauth_clients.client_type` (`'dcr'` | `'machine'`) is the entire authorization boundary for `client_credentials` — see §8. `oauth_clients.service_user_id` links a machine client to the account its tokens authenticate as, deliberately without a foreign key, matching the `oauth_flows.user_id` precedent so erasure ordering stays simple.

**The archive is opportunistic.** Nothing crawls. It fills only from fetches a user already caused, so it deepens over time at zero additional cost to PSE Edge — which matters because PSE Edge itself serves only limited history.

Rules:

- **Schema is applied by Alembic only**, never `create_all` at runtime. Compose runs a one-shot `migrate` service and `check_schema()` fails loudly at startup if migrations were skipped.
- **A failed archive write must never fail a read.** It is caught broadly — a dead database raises `OSError`, not merely `SQLAlchemyError`.
- Postgres imports are **lazy**, inside `build_storage()`. A test asserts SQLAlchemy does not leak into `sys.modules` on `import pse_edge_mcp.server`.

## 10. Extending the server

### Adding a tool to an existing domain

1. Add a method to the relevant repository in `repositories.py`. It owns the cache key, the `FreezeService.get()` call, the parse, and the model.

2. Add a thin tool in `server.py` : validate arguments, call the repository, `return await reply(run)` . Keep the `-> dict[str, Any]` annotation.
3. Add a Pydantic model in `models.py` if the shape is new.
4. Record a fixture in `tests/fixtures/` and test the repository with a fake source.

## Adding a whole data domain

A new domain is **a new repository plus thin tools** — never fetch/parse orchestration in `server.py` . Concretely:

1. Add the narrow Protocol to `sources.py` .
2. Add the fetch method to `client.py` , using the correct dialect (§11).
3. Add the parser to `parsers.py` . Parse by **<thead> label, never column position**.
4. Add the repository, the models, and the tools.
5. Capture a real fixture; tests never touch the network.

## Checklist before opening a PR

- `uv run ruff check .` · `uv run mypy src` · `uv run pytest` all clean
- New endpoint → new fixture, and `docs/endpoints.md` updated
- Shape drift raises `EndpointChangedError` rather than returning partial data
- All reads go through `FreezeService.get()` — **never call `PseEdgeClient` from a tool**
- Bump `version` in `pyproject.toml` and roll `CHANGELOG.md` 's Unreleased section **in the same PR** as the work being released

## Branching

All work lands on `develop` . `develop` reaches `main` only by pull request; `main` is protected. **Merging to `main` publishes a multi-arch image**, and a merge that changes `version` also cuts a GitHub Release and an immutable `<version>` tag. Never commit to `main` directly.

---

## 11. Debugging guide

### Symptom → cause

| Symptom                                                  | Almost certainly                                                                                                                                                                         |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /mcp → <b>421 Invalid Host header</b>               | The SDK's DNS-rebinding guard. <code>PSE_PUBLIC_URL</code> must match the host clients actually use — <code>asgi.transport_security_for()</code> derives the allowlist from it           |
| OAuth completes, then the <b>first tool call fails</b>   | Same as above. Everything about auth is fine; look at the transport layer                                                                                                                |
| <code>MARKET_OPEN_NO_CACHE</code>                        | Working as designed. Retry after the close                                                                                                                                               |
| <code>ENDPOINT_CHANGED</code>                            | PSE Edge changed shape. <b>Re-capture the fixture, compare against <code>docs/endpoints.md</code>, fix the parser.</b> Never paper over it                                               |
| HTTP <b>415</b> from an <code>.ax</code> endpoint        | Wrong dialect — that endpoint needs a JSON body, not form encoding                                                                                                                       |
| Disclosure results in the wrong order                    | <code>sortType</code> must be the literal string <code>"date"</code>                                                                                                                     |
| Signup → <b>503</b>                                      | The mail provider refused. The log names the sender address; ZeptoMail verifies <b>exact</b> domains, so a verified <code>example.com</code> does not cover <code>sub.example.com</code> |
| Passkeys "just don't work"                               | <code>PSE_PUBLIC_URL</code> does not match the browser's origin. Enrolled credentials cannot be migrated, only re-enrolled                                                               |
| App container stopped, <b>empty log</b>                  | A start-time failure, not a crash — a crash always leaves logs. Usually a taken host port, or a bind-mount whose source does not exist                                                   |
| NAS UI shows the project as <b>"Error"</b>               | <code>migrate</code> is a one-shot that exits 0. That is success. Judge health by <code>curl /health</code>                                                                              |
| <code>ImportError</code> on sqlalchemy in a lean install | Something imported Postgres code eagerly. It must be lazy, inside <code>build_storage()</code>                                                                                           |
| POST /oauth/token → <b>unauthorized_client</b>           | The client is not a machine client. Only <code>pse-edge-admin create-machine-client</code> can authorize this grant — a self-registered client never can, by design                      |
| POST /oauth/token → <b>invalid_client</b> (401)          | Wrong secret, unknown client, or a revoked one. All answered identically on purpose, so the endpoint is not an oracle for which client ids exist                                         |
| POST /oauth/token → <b>slow_down</b> (429)               | Rate limit: 20/minute per <code>client_id</code> and per IP. Honour <code>Retry-After</code>                                                                                             |
| A machine client's token stops working early             | <code>revoke-machine-client</code> revokes outstanding tokens too, and revocation lands within <code>PSE_TOKEN_CACHE_TTL</code> (60 s)                                                   |

### Useful commands

```
curl -s https://<host>/health # liveness
curl -s https://<host>/health/ready # readiness (checks DB)
curl -s https://<host>/well-known/oauth-protected-resource # is PSE_PUBLIC_URL right?
curl -sS -o /dev/null -w '%{http_code}\n' -X POST https://<host>/mcp # expect 401

docker compose -f compose.nas.yaml logs -f app
docker compose -f compose.nas.yaml exec app pse-edge-admin list-users
docker compose -f compose.nas.yaml exec app pse-edge-admin list-machine-clients
```

## Reading the log

Every line is timestamped (ISO-8601 with an offset) in both formats; `PSE_LOG_JSON=1` gives one JSON object per line for a log shipper. Secrets are redacted in both formats as a backstop — nothing here logs a credential deliberately.

The lines worth knowing by sight:

| Line                                                                            | Means                                                                                  |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>starting pse-edge-mcp &lt;version&gt; ...</code>                          | the config the process <b>actually</b> resolved to. First thing to read in an incident |
| <code>upstream: fetching from PSE Edge key=...</code>                           | a real request left for PSE Edge                                                       |
| <code>upstream: fetched and cached key=... duration_ms=...</code>               | it came back, and how slow it was                                                      |
| <code>freeze: refusing an uncached read while the market is open</code>         | the policy working, not a fault                                                        |
| <code>auth: rejected a bearer token / quota: refused</code>                     | refusals only — successes are the access log                                           |
| <code>oauth: issued a client_credentials access token</code>                    | a machine client minted a token                                                        |
| <code>oauth: REFRESH TOKEN REUSE detected (WARNING)</code>                      | a rotated token was presented twice, so it <b>leaked</b>                               |
| <code>oauth: DENIED client_credentials to a non-machine client (WARNING)</code> | a misconfigured integration, or someone probing the gate                               |

`upstream: fetching during market hours means the freeze invariant is broken.` Those lines should be a trickle after each 15:00 Manila close and absent between 09:30 and 15:00 on a trading day. That is the single most useful thing to grep this log for.

## Two PSE Edge dialects

Getting this wrong produces an HTTP 415 that looks like a server fault:

|                                                                                                                                                           |                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JSON dialect                                                                                                                                              | POST /common/DisclosureCht.ax<br>Content-Type: application/json<br>→ JSON body. Form encoding gives 415.                                                                           |
| Form dialect                                                                                                                                              | POST /announcements/search.ax<br>POST /companyDisclosures/search.ax<br>POST /keyword/search.ax<br>Content-Type: application/x-www-form-urlencoded<br>→ an HTML FRAGMENT, not JSON. |
| Wire dates are MM-dd-yyyy. PSE Edge's parameter names sometimes differ from its own HTML form field names – trust docs/endpoints.md over the page source. |                                                                                                                                                                                    |

## 12. Testing

**Tests never touch PSE Edge.** All HTTP is mocked with `respx` against 15 fixtures in `tests/fixtures/`, recorded from real captures. A new endpoint requires a new fixture.

| Layer            | How it is tested                                                                          |
|------------------|-------------------------------------------------------------------------------------------|
| Repositories     | Five-line fake sources — no HTTP mocking ( <code>test_repositories.py</code> )            |
| Client / parsers | <code>respx</code> + recorded fixtures                                                    |
| Freeze policy    | Injected calendar pinning the clock ( <code>test_service.py</code> )                      |
| Postgres         | testcontainers, marked <code>@pytest.mark.postgres</code>                                 |
| Auth             | <code>test_auth_journey.py</code> — soft-webauthn, <b>real signatures</b> , real Postgres |
| Deployment       | <code>test_deploy.py</code> — health probes, logging, the real ASGI factory               |

## A lesson worth internalizing

250 passing tests once missed a bug that made the server unusable in production: the transport-security setting. The journey test passed `transport_security=...` **explicitly**, so it exercised a configuration production never built — the fixture worked around the exact defect it should have caught.

**A fixture that configures something production leaves at its default is a blind spot, not a test.** Where practical, test through `create_app()` — the real factory — rather than hand-assembling the stack.

## 13. Configuration reference

All settings live in `config.py` as a frozen dataclass; environment variables override.

| Variable                                  | Default                 | Notes                                       |
|-------------------------------------------|-------------------------|---------------------------------------------|
| PSE_EDGE_BASE_URL                         | https://edge.pse.com.ph |                                             |
| PSE_MARKET_OPEN / PSE_MARKET_CLOSE        | 09:30 / 15:00           | Asia/Manila; drives the freeze              |
| PSE_THROTTLE_RPS / PSE_THROTTLE_BURST     | 1.0 / 2                 | <b>Outbound</b> politeness                  |
| PSE_TIMEOUT_SEC / PSE_RETRY_ATTEMPTS      | 20.0 / 3                |                                             |
| DATABASE_URL                              | unset                   | Unset ⇒ in-memory + no archive              |
| PSE_DB_POOL_SIZE / PSE_DB_MAX_OVERFLOW    | 5 / 10                  |                                             |
| PSE_AUTH_REQUIRED                         | false                   | Requires DATABASE_URL                       |
| PSE_TOKEN_CACHE_TTL                       | 60                      | <b>The revocation-latency budget</b>        |
| PSE_QUOTA_PER_MIN / PSE_QUOTA_PER_DAY     | 60 / 2000               | Per user, <b>in-process</b>                 |
| PSE_PUBLIC_URL                            | http://localhost:8000   | <b>The highest-risk setting — see below</b> |
| PSE_ACCESS_TTL_MIN / PSE_REFRESH_TTL_DAYS | 30 / 30                 |                                             |
| ZEPTOMAIL_API_KEY                         | unset                   | Unset ⇒ emails logged to console            |
| PSE_EMAIL_FROM                            | no-reply@localhost      | Must be a <b>verified</b> sender domain     |
| PSE_USAGE_RETENTION_DAYS                  | 90                      | The privacy page states 90                  |
| PSE_STATEFUL / PSE_SSE                    | false / false           | See below                                   |
| PSE_LOG_JSON / PSE_LOG_LEVEL              | false / INFO            |                                             |

**PSE\_PUBLIC\_URL must be the real external HTTPS URL.** It simultaneously drives the WebAuthn `rp_id`, the links in verification emails, the OAuth issuer in discovery documents, and the DNS-rebinding host allowlist. A wrong value breaks passkeys in a way that looks like a browser bug, and enrolled credentials cannot be recovered — only re-enrolled. Verify it after every deploy: `curl -s https://<host>/.well-known/oauth-protected-resource`

## Why HTTP is stateless with plain JSON by default

The server is 12 read-only tools over data the freeze holds still. It uses **none** of the features MCP sessions exist to enable — no notifications, no resource subscriptions, no sampling, no elicitation, no progress. Every request is self-contained.

That makes horizontal scaling ordinary: any replica serves any request behind plain round-robin — no sticky routing, no per-session memory, no event store. Without SSE, idle clients hold no connection, so N users stop meaning N concurrent connections.

`--stateful` and `--sse` restore session mode for anyone who needs resumability or server-initiated messages. They are independent flags: statelessness is the scaling property, plain JSON is what keeps ordinary proxies out of the way.

**Workers are processes.** All in-memory state — quota windows, the parse memo, the token cache, the usage buffer — is **per worker**, so N workers means a user's effective quota ceiling is up to N× nominal. Scale workers for CPU, and scale limits with them.

## 14. Deployment topologies

### A. VPS that owns ports 80/443

compose.prod.yaml

internet  
| :80 / :443  
▼

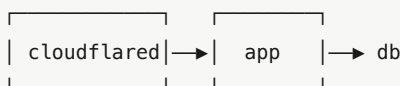
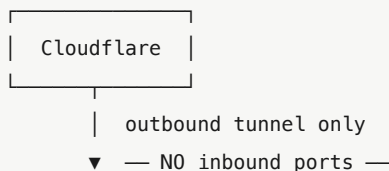


Requires the router to forward  
80→8280 and 443→8243: ACME  
validates on 80/443 only.

### B. NAS behind Cloudflare Tunnel

compose.nas.yaml [+ compose.tunnel.yaml]

internet  
|  
▼ (Cloudflare edge, TLS ends here)



Needs no inbound ports at all.  
Tunnel route is HTTP → app:8000  
(container port, not the host port)

Both pull the published, CI-gated multi-arch image; neither builds from source. Pin `PSE_IMAGE_TAG` — a moving `:latest` that lags the compose file reads as a broken deployment.

The runtime image contains only what the app needs to run: multi-stage, non-root, no build tools, no dev dependencies, no source tree, no credentials in any layer. `scripts/check_image.py` asserts installed distributions **equal** the resolved runtime closure from `uv export`, so any stray package fails the build. Size is reported as information only — **the gate is necessity, not a number**.

## 15. Gotchas

Findings that cost real debugging time. Most are recorded in `docs/endpoints.md` with the evidence.

- **Parse disclosure tables by `<thead>` label, never by column position.** The two search endpoints order their columns differently.
- **Financial-report units labels contradict each other** between the annual and quarterly sections. Values are passed through **verbatim and never rescaled**; each period reports its own `currency_units` for the caller to check.
- **Index changes are printed unsigned.** PSE Edge shows direction only as a colour and a ▲/▼ glyph, so signs are derived from the glyph.
- **selectolax traps:** `iter()` walks direct children only, and `css("a, b")` returns matches **grouped by selector, not in document order** — which once filed all four financial statements under the last heading. Use `root.traverse()`.
- **Accounting negatives** `(1,234)` parse to `-1234`.
- **dividends\_and\_rights\_form.do is an empty shell.** It posts to `dividends_and_rights_list.ax` once per tab, with `DividendsOrRights` in the query string and `cmpy_id` in the body.
- **Homepage feeds are keyed by PSE Edge's own group labels**, not invented buckets.
- **market\_calendar.PSE\_HOLIDAYS is a yearly-maintained seed.** Verify it against the PSE holiday circular each December and whenever touching calendar logic.



- `/health` **must never touch the database**. A DB-dependent liveness probe restarts every replica during a blip, turning a recoverable outage into an outage plus a restart storm. `/health/ready` is where the DB check belongs.

---

## 16. Where to look next

| I want to...                       | Read                                                                                   |
|------------------------------------|----------------------------------------------------------------------------------------|
| Understand why a decision was made | <code>docs/plan.md</code>                                                              |
| Add or fix an upstream call        | <code>docs/endpoints.md</code> , then <code>client.py</code> + <code>parsers.py</code> |
| Change what a tool returns         | <code>models.py</code> , then the repository                                           |
| Change caching behaviour           | <code>service.py</code> — and re-read §4 first                                         |
| Deploy or operate it               | <code>docs/deploy.md</code>                                                            |
| Know the invariants quickly        | <code>CLAUDE.md</code>                                                                 |